

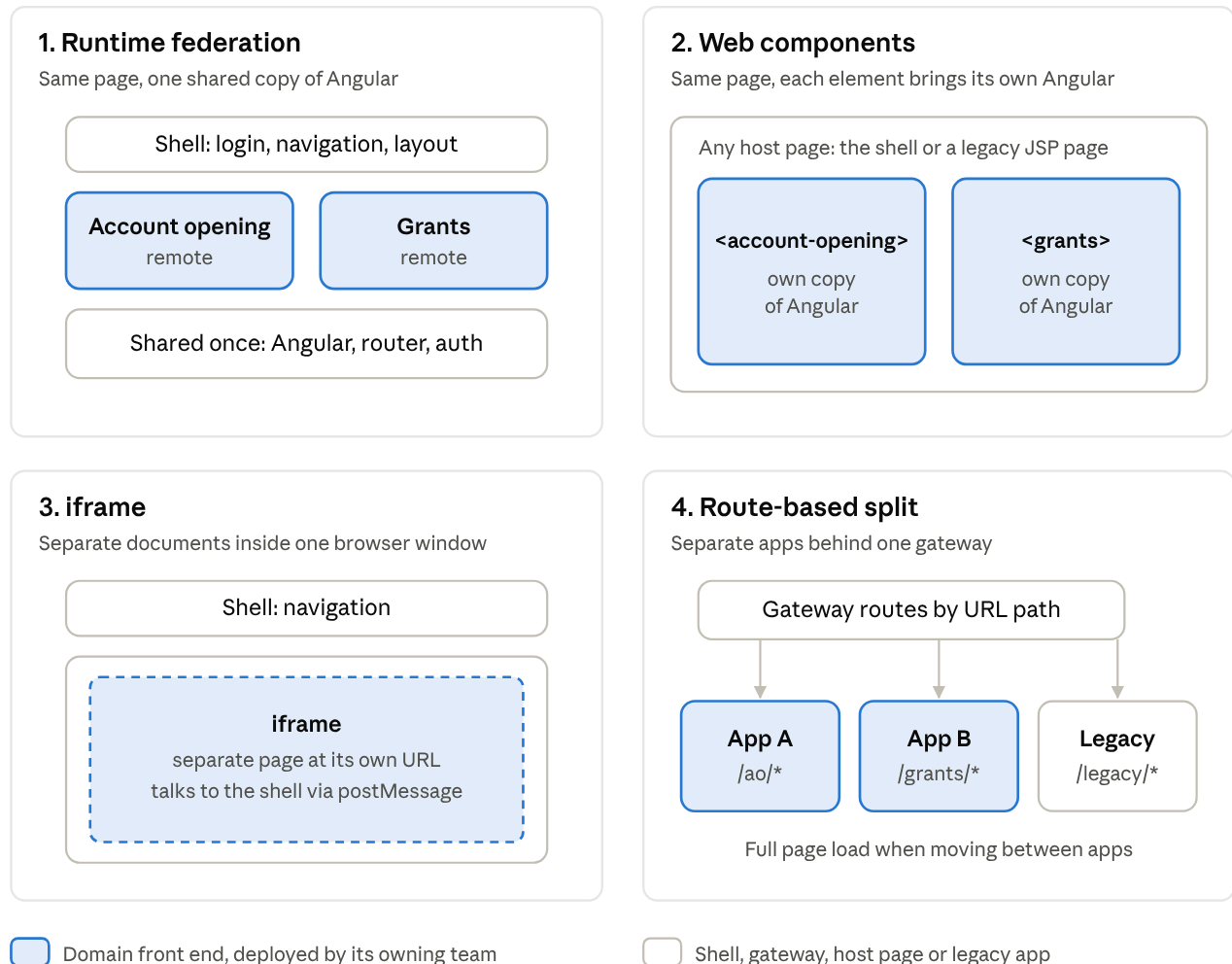
Micro Frontend Patterns: A Guide for Backend Engineers

Oct 6, 2026 · @Siva

Micro frontends reuse ideas backend engineers already know (gateways, plugins, separate processes) and apply them in the browser. This guide explains the four main patterns in those terms, so you can lead a design conversation about them for the app.

At a glance

Each pattern combines separately deployed pieces in a different place



four micro frontend patterns · where the pieces come together

Federation and web components combine pieces inside one page; an iframe keeps each piece in its own document; a route-based split keeps them as separate apps behind the gateway.

1. Angular basics for a backend engineer

An Angular app is a bundle of JavaScript files that runs in the user's browser. The server sends a nearly empty HTML page plus those files. From then on the browser builds every screen itself and calls REST APIs for data.

This is a single-page application (SPA). Clicking around does not fetch new HTML pages. That is the big difference from JSP, where every click is a server round trip that renders a new page on the server.

Angular concept	Spring / backend equivalent	What it does
Component	Controller plus its view template	One piece of a screen (a form, a table) with its own logic
Service and dependency injection	<code>@Service</code> beans and <code>@Autowired</code>	Shared logic injected where needed. A service can be a singleton for the whole app
Router	<code>@RequestMapping</code>	Maps a URL such as <code>/account-opening/step-2</code> to a component, but inside the browser
Lazy loading	Loading a jar only on first use	A section's code is downloaded only when the user first visits that route
HTTP interceptor	Servlet filter	Runs on every outgoing API call, for example to attach the auth token
<code>ng build</code>	<code>mvn package</code>	Compiles TypeScript into optimized JavaScript bundles
npm package (library)	Maven dependency	Reusable code compiled into your app at build time

Angular concept	Spring / backend equivalent	What it does
<code>package.json</code>	<code>pom.xml</code>	Declares dependencies and their versions

Three facts make the rest of this guide easier:

- **Code runs on the user's machine.** A bug or a slow download is felt directly by the user, and you cannot hot-fix one user's browser. That is why bundle size and version mismatches matter more than on a server.
- **The browser has security walls.** Code from one origin (scheme + domain + port) cannot freely read cookies or pages from another origin. Several patterns below are shaped by this rule.
- **Everything on one page shares one JavaScript runtime.** It is like many classes in one JVM: shared memory, shared global state, and one copy of each loaded library unless you load it twice.

2. The baseline: libraries, and what micro frontends change

A library is build-time integration: the same model as a Maven dependency inside a fat jar. Every micro frontend pattern is a way to move that integration from build time to runtime.

How a library change reaches users:

1. The owning team changes its library and publishes a new version, say 1.4.1, to the package registry.
2. Someone bumps the version in the app's `package.json`.
3. The app is rebuilt into one bundle that contains every domain.
4. The whole app is tested and released in its next release window.

The upside is safety. The compiler checks all the code together, there is exactly one version of everything, and a release is atomic. The downside is coupling: no domain reaches users faster than the app's release.

What micro frontends change: each domain builds and deploys its own bundle to its own location. Something then puts those bundles together while the app runs. The four patterns differ only in where that happens:

Pattern	Where the pieces come together	Backend analogy
1. Runtime federation	In the browser, inside one page, sharing one Angular	Plugins loaded at runtime that share a parent classloader
2. Web components	In the browser, inside one page, each with its own Angular	Self-contained fat jars with an embedded server
3. iframe	In the browser, in separate documents	Separate processes that talk by message passing
4. Route-based split	At the gateway, as separate pages	API gateway path routing to separate services

The closer together the pieces live, the smoother it feels to users and the more the teams must coordinate (shared versions, shared rules). The further apart, the more independent and isolated, and the clunkier for users.

3. Pattern 1: Runtime federation

A thin Angular app called the **shell** (or host) loads each domain's code from a different URL while the app runs, and all of them share one copy of Angular. To the user it is one seamless app. Each domain's separately deployed front end is called a **remote**.

Backend analogy: an app server that loads separately deployed plugins at runtime, with Spring and logging loaded once by a parent classloader and shared by every plugin. Think OSGi bundles: deployed independently, running in one JVM.

Tools: Module Federation came from webpack, the older Angular build tool. Native Federation does the same job on the newer esbuild-based Angular build and is the usual choice for recent Angular versions.

How a request flows

1. The user opens the app. The browser downloads the shell, which holds the header, navigation, login handling and the router.
2. The shell reads a **manifest**: a small JSON file listing each remote and where its code lives, for example `"account-opening": "https://cdn/.../account-opening/remoteEntry.json"`. It plays the role of a service registry for UI code, and there is one per environment.

3. The user clicks *Open account*. The shell's router sees that `/account-opening` belongs to a remote.
4. The shell downloads that remote's entry file. It declares what it needs, for example Angular 20 and RxJS 7. The shell already has those loaded, so they are shared, not downloaded again.
5. The account-opening screens render inside the shell's layout, as if they were part of the same app.

What it looks like in code (just enough to follow a conversation)

The remote declares what it exposes, much like a service publishing an interface:

```
// federation.config.js in the account-opening project
exposes: { './routes': './src/app/account-opening.routes.ts' }
```

The shell lazy-loads it on a route, much like resolving a service from a registry:

```
{ path: 'account-opening',
  loadChildren: () => loadRemoteModule('account-opening', './routes').then(m
=> m.ROUTES) }
```

Deploying

The account-opening team deploys new JavaScript files to its own hosting location. The next time any user opens that route, they get the new version. The shell is not rebuilt or redeployed. Rolling back means pointing the manifest at the previous version, or redeploying the old files.

Login and API calls

Everything runs in one browser page, so the shell provides one auth service (a singleton, like a Spring bean). The user logs in once through the shell. Every remote's API calls pass through a shared HTTP interceptor that attaches the token. Remotes never implement login themselves.

How domains talk to each other

Mainly through the URL (navigate to `/grants?donor=123`) and simple browser events such as `accountCreated`. Avoid a shared state store across remotes: it is the front-end version of two microservices writing to the same database table.

The main risk: version mismatch at runtime

If the shell runs Angular 20 and a remote was built against Angular 19, it can break in the user's browser. This is the front-end equivalent of a `NoSuchMethodError` when two jars expect different versions of a shared library, except it fails on a user's machine instead of a server. The fix is governance:

- Share only a few libraries: the framework, the router, auth, the design system.
- All domains upgrade Angular major versions together, on an agreed schedule.
- CI checks that each remote's shared versions match the shell's.

Summary

Good	Seamless single-app experience; smallest downloads; independent deploys; one login
Watch out	Version coordination; manifests and hosting per domain; testing remotes together; cannot host legacy JSP pages
Best fit	An all-Angular app with several teams that need to release independently

4. Pattern 2: Web components (Angular Elements)

Each domain packages its UI as a **custom HTML tag**, for example `<account-opening-app donor-id="123">`. Any web page can drop that tag in, whether it is an Angular shell, a React app or a server-rendered JSP page. The browser itself knows how to run custom tags, because web components are a browser standard, not an Angular feature.

Backend analogy: a Spring Boot fat jar with an embedded Tomcat. It runs anywhere because it carries its own runtime, and that is also why it is heavier.

How it works

1. The domain team builds its Angular code with Angular Elements, which wraps it as a standard browser custom element.
2. They publish one JavaScript file to their hosting location.

3. A host page adds a `<script>` tag pointing at that file, plus the custom tag where the UI should appear.
4. The browser sees the tag, and the domain's code takes over that area of the page.

The contract is explicit, like a small REST interface:

```
<account-opening-app donor-id="123" mode="rep"></account-opening-app>  
<!-- data in: attributes or properties    results out: DOM events, e.g.  
accountCreated -->
```

Deploying

The same as federation: replace the JavaScript file, and every host page gets the new version on its next load.

Login and API calls

If the tag sits inside the Angular shell, it can use the shell's auth service, as in pattern 1. If it sits on a legacy JSP page, it relies on the session cookie the legacy page already uses, or the host page passes a token in as a property.

Why it matters for the app

It is the only pattern that can put new Angular UI **inside an existing legacy page**. That lets you modernize part of a JSP page (one panel, one widget) before the whole page moves. During a strangler migration that can be a real accelerator.

Summary

Good	Works in any host, including legacy pages; versions can differ per domain; clear input/output contract
Watch out	Each element bundles its own Angular, so users download more; several Angular copies on one page use more memory; styles need care to avoid clashes
Best fit	New UI that must live inside legacy pages, or a widget shared with apps built on other frameworks

5. Pattern 3: iframe

The shell displays a box (an iframe) that loads a completely separate web page from its own URL. It is a browser window inside the page.

Backend analogy: two separate processes, like two JVMs, that share no memory and no classes and only talk by passing messages.

How it works

1. The shell renders its navigation plus `<iframe src="https://app.example/legacy/accountOpening.jsp">`.
2. The browser loads that page fully separately: its own HTML, scripts, styles and memory.
3. The shell and the framed page talk only through `postMessage`, which is like sending small JSON messages over a queue. For example, the framed page posts "account created, id 456" and the shell navigates to the next screen.

Login and the same-origin rule

- **Same domain as the shell:** the framed page shares cookies with the shell, so an existing session just works. This is the easy case.
- **Different domain:** browsers increasingly block third-party cookies inside iframes. You then need a token hand-off through `postMessage` or a silent sign-in, which is fiddly and easy to get wrong.

If you use iframes, keep them on the same origin as the shell, for example by serving legacy pages under `/legacy/*` through the gateway.

Common problems

- **Sizing:** the frame does not grow with its content, so you get scrollbars inside scrollbars unless you add resize messaging.
- **Navigation:** the browser back button and deep links get confusing, because the frame has its own history.
- **Look and feel:** the framed page keeps its own styles, so it never quite matches the shell.
- **Accessibility:** screen readers and keyboard navigation handle frames poorly.
- **Performance:** each frame loads everything separately.

Summary

Good	Strongest isolation (a crash or bad CSS inside cannot break the shell); any technology inside; quickest way to show a legacy page in a new shell without changing it
Watch out	Sizing, deep links, back button, styling, accessibility, slower loads; cross-domain cookies
Best fit	A temporary bridge to legacy pages during migration, rarely the long-term design

6. Pattern 4: Route-based split (gateway or reverse proxy)

Each domain is a fully separate web app, and the gateway sends each URL path to the right one. `/account-opening/*` goes to the account-opening app, `/grants/*` to the grants app, and `/legacy/*` to the existing Spring Boot app. Moving between domains is a normal full-page navigation.

Backend analogy: exactly API gateway path routing for microservices (`location /orders` → orders service in nginx), applied to whole web pages. This is the pattern you already know best.

How it works

1. The user goes to `/account-opening`. The gateway serves the account-opening app's HTML and JavaScript.
2. The user clicks a link to `/grants`. The browser leaves that app entirely and loads the grants app: a full page reload.
3. Each app renders its own copy of the header and navigation, ideally from a shared library so they look identical.

Login

The session lives in a cookie on the shared domain, often set by a backend-for-frontend (BFF) or the gateway. Because every app sits under the same domain, the cookie travels with every page and the user stays logged in. It is the same mechanism the legacy JSP pages use today.

Common problems

- **Page reloads:** moving between domains feels less smooth and takes longer than an in-app transition.
- **Duplicated chrome:** header, navigation and login checks exist in every app, so they must come from a shared library to stay consistent.
- **In-progress work:** a half-filled form is lost on navigation unless it is saved first, for example as a server-side draft.

Summary

Good	Simplest pattern; no runtime code sharing; fully independent apps; legacy pages slot in as just another route
Watch out	Page reloads between domains; duplicated shell work; state does not carry across
Best fit	Domains users rarely switch between mid-task, and routing to legacy pages during migration

7. Other approaches you may hear about

Two more names come up in these conversations. Neither is a strong fit for an all-Angular business app, but you should recognize them.

- **single-spa:** a JavaScript orchestrator that mounts different apps on one page by route, often used to mix React, Angular and Vue. Thoughtworks warns against mixing frameworks without a deliberate transition plan ("micro frontend anarchy"), so it adds little when every team uses Angular.
- **Server-side composition (SSI, ESI, edge includes):** a server or CDN stitches HTML fragments from different services into one page before sending it. It is common on content and e-commerce sites, and rare for interactive app UIs like ours.

8. Side-by-side comparison

Federation gives the best experience with the most coordination; route-based and iframe give the most isolation with the clunkiest experience.

	1. Federation	2. Web components	3. iframe	4. Route-based
Backend analogy	Plugins sharing a parent classloader	Fat jar with embedded server	Separate processes, message passing	Gateway path routing
Where pieces combine	Browser, same page	Browser, same page	Browser, separate documents	Gateway, separate pages
User experience	Seamless	Seamless	Clunky (frames)	Page reloads
Angular copies	One, shared	One per domain	One per frame	One per app
Isolation	Low	Medium	Strong	Strong
Framework versions	Must match	Can differ	Can differ	Can differ
Login	Shared auth service	Shared service or cookie	Cookie if same origin, else hand-off	Shared domain cookie
Can show legacy JSP?	No	Yes, inside a page	Yes, wraps the page	Yes, as a route
Setup effort	Medium	Medium	Low	Low
Long-term fit	Main pattern for new Angular domains	Niche: UI inside legacy or other frameworks	Temporary bridge	Bridge, or loosely related domains

9. How login works in each pattern

The rule for every pattern: the user signs in once, the shell or gateway owns the session, and domains never build their own login.

Typical login flow with a shell (patterns 1 and 2)

1. The user opens the app. The shell finds no session and redirects to the identity provider (single sign-on).
2. The user signs in there. The identity provider redirects back to the shell with a one-time code.
3. The shell exchanges the code for tokens. The common standard is OpenID Connect (OIDC) with the authorization code flow and PKCE, which protects the code exchange in a browser.
4. The shell keeps the token in memory, inside its auth service, and refreshes it before it expires.
5. When a remote calls its API, the shared HTTP interceptor adds `Authorization: Bearer <token>`. The API gateway or the backend validates it.
6. Logout clears the shell's session and the identity provider's session together.

A safer variant, the backend-for-frontend (BFF): a thin server holds the tokens and gives the browser only an `HttpOnly` session cookie, which JavaScript cannot read. API calls go through the BFF, which attaches the token server-side. This keeps tokens away from malicious scripts, and many security teams prefer it.

Per pattern

Pattern	How domains get the session	Note
1. Federation	Shared auth service and interceptor from the shell	Simplest, because everything is one page
2. Web components	Same as federation inside the shell; inside a legacy page, the existing cookie or a token passed in	Agree who owns refresh
3. iframe	Same-origin cookie; otherwise a token sent by <code>postMessage</code>	Keep frames same-origin to avoid third-party cookie blocks
4. Route-based	Shared cookie on the common domain, usually set by a BFF or the gateway	Same mechanism as the legacy JSP pages today

Carrying the session over to legacy pages

Users must never sign in twice when they move between new screens and legacy pages. Three common ways, best first:

- **One domain, one session cookie:** serve legacy pages under the same domain through the gateway, for example `/legacy/*`, and have both sides trust the same session (a BFF cookie or an SSO session).
- **Token exchange:** the gateway turns the new app's token into whatever the legacy app expects, such as a legacy session or header.
- **Silent SSO:** if the legacy app already uses the same identity provider, it signs the user in invisibly when they arrive.

Whichever you choose, authorization stays in the backend. Route guards in the shell only hide screens; they are not security.

10. Applying it to the app

If micro frontends are chosen, the usual answer for an app mid-migration is a deliberate mix: one main pattern for new domains and a simpler one to keep legacy pages reachable until they are retired.

Need	Pattern	Why
New Angular domains (account opening, contributions, grants)	1. Federation	One seamless app, shared Angular, independent deploys
Legacy JSP pages not yet migrated	4. Route-based, or 3. iframe same-origin	Reachable without rework; the shell's route map decides new vs legacy behind a feature flag
New UI inside a legacy page before it moves	2. Web components	Only pattern that can live inside a JSP page

How a click is routed during the migration:

1. The shell holds a route map: each URL pattern points to a remote or to legacy, controlled by a feature flag.
2. A migrated route loads its remote inside the shell.

3. A legacy route goes through the gateway to the Spring Boot app (for example `/legacy/*` on the same domain), carrying the same session, so there is no second login.
4. When a remote reaches parity with its legacy page, the flag flips and the legacy route is retired.

Remember the alternative. The same domain slices can instead be built as libraries and installed into one app (see the companion deck). A slice built as a library with a clean route entry point can later be wrapped as a remote, so starting with libraries does not close the door.

A caveat that applies to every pattern: if the domain APIs still deploy as one backend release, front-end independence only helps with UI-only changes.

11. Leading the conversation

Open with the framing, not the tools: "Every option answers one question: where do separately deployed pieces get put together? At the gateway, in a frame, or in the same page. The closer together, the smoother for users and the more coordination for teams."

Questions to ask the room

1. How often do users move between domains within one task? Often favors federation; rarely means route-based is enough.
2. Do we need new UI inside legacy pages before they move? If yes, web components have a role.
3. Can every team agree to upgrade Angular together? If not, federation becomes risky.
4. How does login work today, and is everything on one domain? That decides how easy the session sharing is.
5. Can backend APIs deploy per domain? If not, front-end independence only helps with UI-only changes.
6. How fast can the app release today? If it can release daily with automated tests, libraries may be enough.

Pushback you are likely to hear

They say	You can answer
"Isn't this over-engineering?"	It can be. If the app can release often with automated regression, libraries may be enough. That is why the proof of concept measures both.
"Why not iframes? They're simple."	They are a good temporary bridge to legacy pages, but sizing, back-button and accessibility problems make them a poor long-term home.
"What happens when one domain breaks?"	With federation the shell shows a fallback for that area, the rest keeps working, and the owning team rolls back alone. With a library, a defect blocks the whole app release.
"Won't it be slower?"	Federation shares Angular, so the overhead is small. Web components and iframes cost more. We will measure load time in the proof of concept instead of guessing.
"Who owns the shell?"	A small platform group. The shell stays thin: login, navigation, layout, flags, monitoring, and no business logic.
"Can teams pick their own framework?"	No. One framework and one design system for all; mixing frameworks is what Thoughtworks calls micro frontend anarchy.
"How do we test it all together?"	Each domain has its own tests, contract tests check the shell-to-domain interface, and a short end-to-end smoke test runs in the shell on every deploy.

Vocabulary

Term	Meaning
Shell / host	The app that loads the other pieces and owns login, navigation and layout
Remote	A domain's separately deployed front end

Term	Meaning
Manifest	Per-environment config listing where each remote lives
Shared singletons	Libraries loaded once and used by all pieces (Angular, router, auth)
Version skew	Pieces built against different library versions meeting at runtime
Native / Module Federation	The tools that load remotes at runtime in Angular (esbuild / webpack builds)
Angular Elements	The Angular tool that packages a component as a web component
postMessage	The browser API that lets a page and an iframe send each other messages
Same origin	Same scheme, domain and port; the boundary for cookies and scripts
BFF (backend-for-frontend)	A thin server that holds the session and tokens for the UI
OIDC with PKCE	The standard browser-safe sign-in flow with an identity provider

Sources

- [Thoughtworks Radar: Micro frontends](#)
- [Thoughtworks Radar: Micro frontend anarchy](#)
- [Angular blog: Micro Frontends with Angular and Native Federation](#)
- [Nx: Micro frontend architecture](#)